

# Assignment: Solving a System of Linear Equations With Gauss Elimination

## Academic Integrity Declaration

Check the statement that applies, and sign your name.

| Check | Statement                                                                                                                      |
|-------|--------------------------------------------------------------------------------------------------------------------------------|
|       | I have <b>NOT</b> used LLM (ChatGPT, etc), any online materials, or other students' reports/codes to complete this assignment. |
| V     | I have <b>NOT completed this assignment entirely on my own.</b>                                                                |

If you received any assistance **that violates the class policy**, you must explain it in detail. A penalty will be applied depending on the **extent and nature of the assistance**.

Failure to disclose the assistance, you will get Grade F.

- *Partial pivoting* 정확한 기준을 모르겠어서 인터넷 찾아봄
- 참고한 사이트: <https://blog.naver.com/cj3024/221114187257>

By signing below, I confirm that I have honestly and truthfully answered the academic integrity declaration.

Date: 2025.10.23

Sign: 유채은

## What you need to submit

Submit as a zip file: [Assignment\\_GaussElim\\_YuChaeun\\_22200476.zip](#)

- **Report** (pdf)
- **Src Codes:**   (1) [Assignment\\_GaussElim\\_YuChaeun\\_22200476.cpp](#)  
                           (2) [myNP\\_22200476.h](#), [myNP\\_22200476.cpp](#)  
                           (3) [myMatrix\\_22200476.h](#), [myMatrix\\_22200476.cpp](#)
- Review how to use structure type for using 2D array
- Review [how to use matrix structure](#),   ([Tutorial Video](#))
- You should insert exceptional/error handling  
   e.g. giving error message when square matrix is not used, div by zero etc.  
   \*We will only consider square matrix A (n by n) for this assignment.

### Datafile:

[Directory]

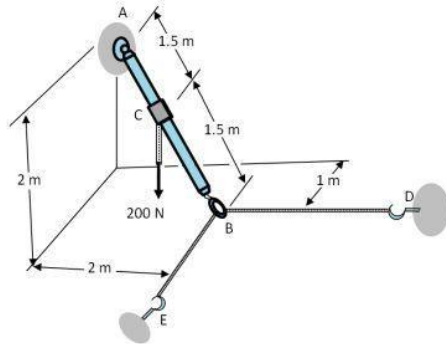
\NP\NP\_Data \Assignment5\

[File Name] ( \*.txt)

| Question No. | Variable Name        | File Name                        |
|--------------|----------------------|----------------------------------|
| Q1           | matrix A<br>vector b | prob1_matA.txt<br>prob1_vecb.txt |
| Q2           | matrix A<br>vector b | prob2_matK.txt<br>prob2_vecf.txt |

**Problem: Solve the following linear systems**

**Q1. Determine the reactions at the ball joint and tension forces in the following system.**



$$\sum \mathbf{F} = 0, \sum \mathbf{M}_A = 0$$

$$A_x + T_E = 0$$

$$A_y + T_D = 0$$

$$A_z - 200 = 0$$

$$T_D - 2T_E = 0$$

$$-2T_E + 100 = 0$$

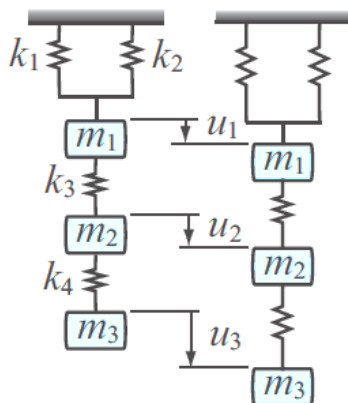
**ANS:  $A_x = -50$ ,  $A_y = -100$ ,  $A_z = 200$ ,  $T_D = 100$ ,  $T_E = 50$**

**Q2. Determine the displacement ( $u_1, u_2, u_3$ ) from  $\mathbf{Ku} = \mathbf{f}$**

They are in the equilibrium states, and  $u_1, u_2, u_3$  are the relative displacement for each mass.

$m_1 = 2 \text{ kg}$ ,  $m_2 = 3 \text{ kg}$ ,  $m_3 = 1.5 \text{ kg}$ ,  $g = 9.81 \text{ m/s}^2$

$k_1 = 30 \text{ [N/m]}$ ,  $k_2 = 25 \text{ [N/m]}$ ,  $k_3 = 20 \text{ [N/m]}$ ,  $k_4 = 15 \text{ [N/m]}$



$$(k_1 + k_2 + k_3)u_1 - k_3u_2 = m_1g$$

$$-k_3u_1 + (k_3 + k_4)u_2 - k_4u_3 = m_2g$$

$$-k_4u_2 + k_4u_3 = m_3g$$

**ANS:  $u_1 = 1.16$ ,  $u_2 = 3.37$ ,  $u_3 = 4.35$**

## Procedure

### 1. Gauss elimination method(without partial pivoting [15]

#### 1a) Write a pseudocode for Gauss Elimination [5pt]

---

```

for k = 0 to (n - 1)
    for i = (k + 1) to n
        mult = A_ik / A_kk

        for j = k to n
            U_ij = A_ij - mult * A_kj
        End j
        D_i = d_i - mult * d_k
    End i
End k

```

---

#### 1b) Create a C function for Gauss elimination (without partial pivoting) [10pt]

```
void gaussElim(Matrix A, Matrix b, Matrix U, Matrix d);
```

- Input: matrix **A**(n x n), vector **b**(n x1)

- Output: matrix **U**, vector **d** //  $Ax=b \rightarrow Ux=d$

---

```

void gaussElim(Matrix* A, Matrix* b, Matrix* U, Matrix* d)
{
    // copy matA to matU & vecb to vecd
    *U = copyMat(*A);
    *d = copyMat(*b);

    double mult = 0;
    int n = A->rows;

    for (int k = 0; k < n - 1; k++) {
        for (int i = k + 1; i < n; i++) {

            mult = A->at[i][k] / A->at[k][k];
            for (int j = k; j < n; j++) {
                U->at[i][j] = A->at[i][j] - mult * A->at[k][j];
            }

            d->at[i][0] = d->at[i][0] - mult * d->at[k][0]; // ***** !!!!! mult *
        }
    }
}

```

---

**2. Create a back-substitution function to solve  $Ux=d$  [15]****2a) First, write a pseudocode [5pt]**


---

```

for i = (m - 1) to 0
    sum = 0;

    for j = (i + 1) to n
        sum += U(i, j) * x(j, 0);
    end j

    x(i, 0) = ( d(i, 0) - sum ) / U(i, i);
end i

```

---

**2b) Then, create a C/C++ function for back substitution [10pt]**

```
void backsub(Matrix U, Matrix d, Matrix x);
```

- Input: matrix **U**(n x n), vector **d**(n x1)
  - Output: vector **x** (nx1)
- 

```

void backsub(Matrix* U, Matrix* d, Matrix* x)
{
    int m = U->rows;
    int n = U->cols;

    double sum = 0;

    for (int i = m - 1; i >= 0; i--) {
        sum = 0;

        for (int j = i + 1; j < n; j++) { // n == i 일때 (맨 처음 인덱스) 문제가 생김
                                         //e.g. for(j = 5; j < 5; j++) -> 강 실행 안하고
                                         건너뛴
            sum += U->at[i][j] * x->at[j][0];
        }

        x->at[i][0] = (d->at[i][0] - sum) / U->at[i][i];
    }
}

```

---

### 3. Show the output results for Q1 and Q2 systems [10pt]

#### Q1. output

```
matA =
  1.000000    0.000000    0.000000    0.000000    1.000000
  0.000000    1.000000    0.000000    1.000000    0.000000
  0.000000    0.000000    1.000000    0.000000    0.000000
  0.000000    0.000000    0.000000    1.000000   -2.000000
  0.000000    0.000000    0.000000    0.000000    2.000000

vecb =
  0.000000
  0.000000
 200.000000
  0.000000
 100.000000

U =
  1.000000    0.000000    0.000000    0.000000    1.000000
  0.000000    1.000000    0.000000    1.000000    0.000000
  0.000000    0.000000    1.000000    0.000000    0.000000
  0.000000    0.000000    0.000000    1.000000   -2.000000
  0.000000    0.000000    0.000000    0.000000    2.000000

d =
  0.000000
  0.000000
 200.000000
  0.000000
 100.000000

x =
 -50.000000
 -100.000000
 200.000000
 100.000000
  50.000000

Press any key to continue . . . |
```

#### Q2. output

```
----- Q2 -----
matK =
  75.000000   -20.000000    0.000000
 -20.000000    35.000000   -15.000000
  0.000000   -15.000000    15.000000

vecf =
 19.620001
 29.430000
 14.715000

matV =
  75.000000   -20.000000    0.000000
  0.000000    29.666667   -15.000000
  0.000000    0.000000    7.415730

vec_e =
 19.620001
 34.662001
 32.240731

vec_u =
 1.159364
 3.366614
 4.347614

C:\Users\USER\source\repos\NP\assignment\Assignment_GaussEl
ocess 13160) exited with code -1073741819 (0xc0000005).
Press any key to close this window . . .
```

**4. Compare your answer with MATLAB [5pt]**

Write Matlab code

// your MATLAB code

```

Q1.
clear all, close all, clc ;

A = [ 1 0 0 0 1;
      0 1 0 1 0;
      0 0 1 0 0;
      0 0 0 1 -2;
      0 0 0 0 -2];

b = [ 0;
      0;
      200;
      0;
      -100];

% gauss elimination + backward
substitution
x = A \ b;

Ax = x(1);
Ay = x(2);
Az = x(3);
TD = x(4);
TE = x(5);

fprintf('Ax = %.4f\n', Ax);
fprintf('Ay = %.4f\n', Ay);
fprintf('Az = %.4f\n', Az);
fprintf('T_D = %.4f\n', TD);
fprintf('T_E = %.4f\n', TE);

```

```

Q2.
clear all, close all, clc ;

A = [ 75 -20 0;
      -20 35 -15;
      0 -15 15];

b = [ 19.92;
      29.43;
      14.715];

% gauss elimination + backward
substitution
x = A \ b;

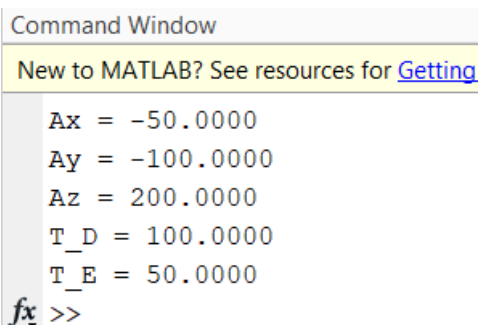
u1 = x(1);
u2 = x(2);
u3 = x(3);

fprintf('u1 = %.4f\n', u1);
fprintf('u2 = %.4f\n', u2);
fprintf('u3 = %.4f\n', u3);

```

// MATLAB output

Q1. Output



Command Window

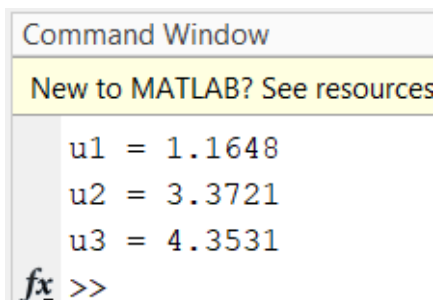
New to MATLAB? See resources for [Getting](#)

```

Ax = -50.0000
Ay = -100.0000
Az = 200.0000
T_D = 100.0000
T_E = 50.0000
fx >>

```

Q2. output



Command Window

New to MATLAB? See resources for [Getting](#)

```

u1 = 1.1648
u2 = 3.3721
u3 = 4.3531
fx >>

```

## Extra-credit (Optional)

will be added partially to overall assignment

### 5. Gauss elimination method (with partial pivoting) [25pt]

Modify your Gauss elimination code to include partial pivoting

```
void gaussElim(Matrix A, Matrix b, Matrix U, Matrix d, Matrix P);
```

- Input: matrix **A**(n x n), vector **b**(n x1)

- Output: matrix **U**, vector **d**, matrix **P** //  $PAx=Pb \rightarrow Ux=d$

#### a) First write a pseudocode [10pt]

---

// your pseudocode

```

for k = 0 to (m - 2)

    big = k
    if ( max = fabs(A(k, k)) )
    for i = k to (m-1)
        if (fabs(A(i, k))
            big = i;
            max = fabs(A(i, k))

    for j = 0 to (n-1)
        temp = P(k, j)
        tmpA = A(k, j)

        P(k, j) = P(big, j)
        A(k, j) = A(big, j)

        P(big, j) = temp;
        A(big, j) = tmpA;

```

---

#### b) Then, create a C/C++ function [10pt]

---

// your code

```

void partialPvt(Matrix* A, Matrix* b, Matrix* P)
{
    // initialization
    int m = A->rows;
    int n = A->cols;
    int big;
    double max, temp, tmpA = 0;

```



```

// determine permutation matrix P
for (int k = 0; k < (m - 1); k++) {

    big = k;
    max = fabs(A->at[k][k]);
    for (int i = k; i < m; i++) {
        if (fabs(A->at[i][k]) > max) {
            big = i;
            max = fabs(A->at[i][k]);
        }
    }
    // row swapping
    for (int j = 0; j < n; j++) {
        temp = P->at[k][j];
        tmpA = A->at[k][j];

        P->at[k][j] = P->at[big][j];
        A->at[k][j] = A->at[big][j];

        P->at[big][j] = temp;
        A->at[big][j] = tmpA;
    }
}

// partial pivoting
*b = multMat(*P, *b);
}

```

---

**c) Validate your code with the test matrix A [5pt]**

$$A = \begin{bmatrix} 1 & 3 & -2 & 4 \\ 2 & -0.1 & 3 & -1 \\ -1 & 7 & -4 & 2 \\ 3 & -1 & 4 & 2 \end{bmatrix}$$

$$P = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad U = \begin{bmatrix} 3 & -1 & 4 & 2 \\ 0 & 6.667 & -2.667 & 2.667 \\ 0 & 0 & -2 & 2 \\ 0 & 0 & 0 & -2 \end{bmatrix}$$

```
----- partial pivoting -----  
testA =  
    3.000000    -1.000000    4.000000    2.000000  
   -1.000000    7.000000   -4.000000    2.000000  
    2.000000   -0.100000    3.000000   -1.000000  
    1.000000    3.000000   -2.000000    4.000000  
  
testU =  
    3.000000    -1.000000    4.000000    2.000000  
    0.000000    6.666667   -2.666667    2.666667  
    0.000000    0.000000    0.560000   -2.560000  
    0.000000    0.000000    0.000000   -7.142857  
  
P =  
    0.000000    0.000000    0.000000    1.000000  
    0.000000    0.000000    1.000000    0.000000  
    0.000000    1.000000    0.000000    0.000000  
    1.000000    0.000000    0.000000    0.000000
```